pontIA

MÁSTER INGENIERÍA DE DATOS

Consultas 4

Window Functions - Recursos

Daniel Trabas

pontIA

Índice

Objetivo 3

¿Cómo se articulan los datos mediante las Window Functions? 3

Anexos 11

PontIA

Objetivo

Las Window Functions (funciones de ventana) se utilizan para realizar cálculos sobre un conjunto de filas de una tabla que están ligadas de alguna manera a la fila actual.

A diferencia de las funciones de agregación convencionales, estas funciones no consolidan ni agrupan las filas de entrada en un único registro de salida. En su lugar, cada fila de la consulta conserva su propia identidad y se muestra individualmente, permitiendo que el motor acceda a información que va más allá de los límites del registro que se está evaluando en ese momento.

¿Cómo se articulan los datos mediante las Window Functions?

- **OVER:** es el elemento obligatorio que distingue sintácticamente a una función de ventana de una función o agregación normal. Esta cláusula le indica al motor de base de datos, exactamente cómo fragmentar y ordenar los datos para su procesamiento.
- **PARTITION BY:** divide las filas lógicas resultantes de la consulta en grupos o particiones independientes que comparten el mismo valor en la columna especificada. El cálculo de la función de ventana se ejecuta de forma aislada para cada grupo y se reinicia a cero al pasar a la siguiente partición. Si se omite esta cláusula, el motor trata a toda la tabla como una única gran partición.
- **ORDER BY:** define el orden secuencial en el que la función de ventana procesa las filas de cada partición. Es importante destacar que esta ordenación es puramente interna para el cálculo (necesaria, por ejemplo, para calcular acumulados cronológicos o números de fila) y no define el orden final en el que se mostrarán los datos al usuario en la salida de la consulta.
- **Marco de la ventana (Window Frame):** para cada registro evaluado, se define un subconjunto de filas dentro de su partición llamado "marco". Ciertas funciones actúan únicamente sobre las filas que entran en este marco dinámico, en lugar de toda la partición. Por ejemplo, al usar ORDER BY, el marco por defecto abarca desde el inicio de la partición hasta el registro actual.
- **Ciclo de ejecución lógica:** las funciones de ventana se ejecutan en una de las etapas finales de la consulta, concretamente después de que se han procesado, agrupado y filtrado los datos mediante WHERE, GROUP BY y HAVING. Esto significa que las funciones de ventana procesan la "tabla virtual" ya depurada por el resto de las cláusulas de la consulta.

SELECT con GROUP BY y filtros HAVING

Filtrado de consultas agrupadas

Antes de proceder a ver las Window Functions, vamos a conocer la forma en la que podemos realizar un filtrado sobre un conjunto de resultados agrupados.

En SQL, la cláusula GROUP BY condensa múltiples filas con idénticos valores en columnas específicas en una única fila de resultado. Así, esta cláusula condensa en una sola fila todas las filas seleccionadas que compartan los mismos valores para las expresiones agrupadas.

Esto significa que la evaluación del WHERE ya ha ocurrido de forma individual para descartar filas físicas antes de la agrupación. Por ello, no es posible aplicar filtros de agregación (por ejemplo, filtrar grupos donde el recuento de registros o la suma sea superior a cierto límite) utilizando WHERE, ya que este último no tiene conocimiento de los grupos.

Para solucionar este requerimiento, PostgreSQL implementa la cláusula HAVING, la cual evalúa condiciones booleanas sobre grupos creados por GROUP BY

Diferencias clave:

- **WHERE:** filtra filas individuales antes de que ocurra la agrupación.
- **HAVING:** filtra los grupos resultantes después de que GROUP BY se haya ejecutado.

Ejemplos prácticos sobre demodb

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

Filtra las rutas de vuelo (bookings.flights) que tienen más de 2 vuelos cancelados registrados para auditoría de calidad de aeropuertos:

```
SELECT  fl.route_no,
        COUNT(fl.flight_id) AS total_cancelados
FROM    bookings.flights fl
WHERE   fl.status = 'Cancelled'
GROUP BY fl.route_no
HAVING COUNT(fl.flight_id) > 2
ORDER BY total_cancelados DESC;
```

Encuentra los pasajeros que han comprado más de 3 billetes individuales (tickets) y cuyo precio promedio por segmento (segments) sea superior a 25.000,00:

```
-- 2. Encuentra los pasajeros que han comprado más de 3, cuyo precio promedio
sea superior a 25.000,00
SELECT tk.passenger_name,
        tk.passenger_id,
        COUNT(DISTINCT tk.ticket_no) AS total_billetes,
        AVG(sg.price) AS precio_promedio
FROM    bookings.tickets tk
  INNER JOIN bookings.segments sg ON tk.ticket_no = sg.ticket_no
GROUP BY tk.passenger_name, tk.passenger_id
HAVING COUNT(DISTINCT tk.ticket_no) > 3 AND AVG(sg.price) > 25000.00
ORDER BY total_billetes DESC;
```

Filtra los modelos de aviones (bookings.airplanes) que tienen más de 100 asientos totales, pero de los cuales más de 20 están configurados específicamente como clase ejecutiva (Business) en la tabla seats:

```sql
-- 3. Encuentra los modelos de aviones (bookings.airplanes) que tienen más de
100 asientos y más de 20 asientos en clase Business
SELECT ap.model,
       COUNT(st.seat_no) AS total_asientos,
       SUM(CASE
             WHEN st.fare_conditions = 'Business' THEN 1
             ELSE 0
           END) AS asientos_business
FROM bookings.airplanes ap
  INNER JOIN bookings.seats st ON ap.airplane_code = st.airplane_code
GROUP BY ap.model
HAVING COUNT(st.seat_no) > 100
  AND SUM(CASE
            WHEN st.fare_conditions = 'Business' THEN 1
            ELSE 0
          END) > 20;
```

SELECT OVER (PARTITION, ORDER) - Window Functions

Una Window Function o función de ventana, se utiliza para realizar un cálculo sobre un conjunto de filas de la tabla que están de alguna manera relacionadas, con la fila actual. Sin embargo, este tipo de funciones, no hacen que las filas se agrupen en una sola fila de salida. Gracias a ello, las filas conservan sus identidades individuales.

Esta propiedad es muy valiosa para realizar análisis interactivos donde se requiere comparar el detalle de la fila actual con métricas del grupo. Como por ejemplo, ver un precio y su desviación sobre el precio medio.

La sintaxis se basa en la palabra clave OVER, que define la ventana de cálculo mediante sus sub-cláusulas:

Sub-cláusula	Detalle de Funcionamiento e Impacto en Plan de Consulta
PARTITION BY	Divide el conjunto de registros de entrada en grupos de control lógico independientes. La función agregada se reinicia para cada partición.
ORDER BY	Define el ordenamiento en el cual la ventana física va a recorrer los registros, lo que habilita cálculos acumulativos (running totals).
Frame Clause	Modifica el marco de filas que se consideran en el cálculo (por ejemplo: ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Ejemplos prácticos sobre demodb

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

Ranking de precios de billetes por clase de tarifa

Utiliza la función de ventana DENSE_RANK() para clasificar los billetes de mayor a menor precio dentro de cada una de las diferentes clases de tarifa (fare_conditions) de manera independiente:

```
-- 1. Ranking de precios de billetes por clase (fare_conditions) utilizando
DENSE_RANK()
SELECT sg.ticket_no,
       sg.flight_id,
       sg.fare_conditions,
       sg.price,
       DENSE_RANK() OVER (
         PARTITION BY sg.fare_conditions
         ORDER BY sg.price DESC
       ) AS ranking_precio_clase
FROM bookings.segments sg
LIMIT 200;
```

Acumulado de ingresos de reservas (Running Total)

Calcula la suma acumulada de los importes de reservas (bookings) a lo largo del tiempo, mostrando tanto el importe individual de la reserva como el saldo acumulado total al momento de esa reserva:

```
-- 2. Ranking de ingresos acumulados por fecha de reserva (book_date)
utilizando SUM() y OVER()
SELECT bk.book_ref,
       bk.book_date,
       bk.total_amount,
       SUM(bk.total_amount) OVER (
         ORDER BY bk.book_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS ingresos_acumulados
FROM bookings.bookings bk
ORDER BY bk.book_date ASC
LIMIT 200;
```

Intervalo de tiempo transcurrido entre vuelos sucesivos

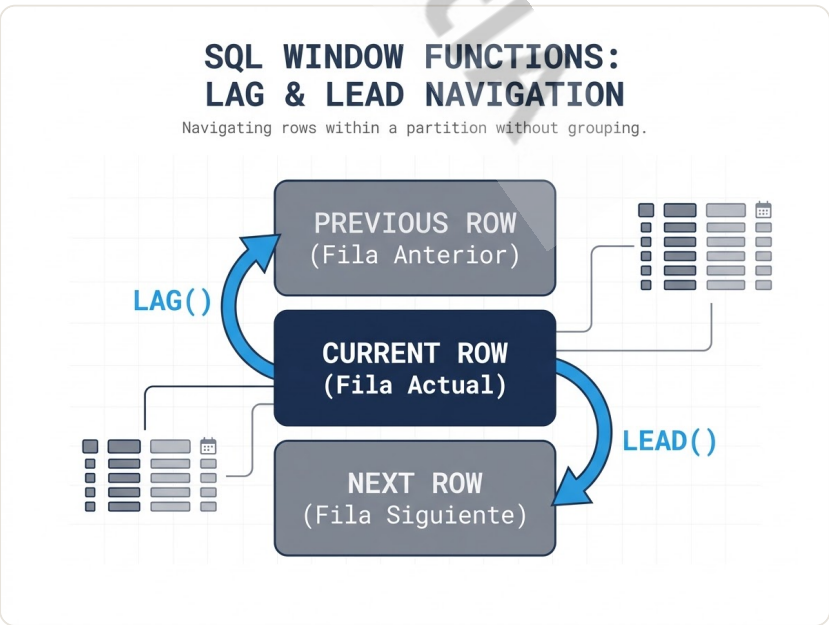
Para cada vuelo completado (Arrived), compara su hora de salida real con el despegue del vuelo inmediatamente anterior en la misma ruta (route_no) para monitorizar demoras o tiempos de respuesta entre aviones:

```sql
-- 3. Ranking de intervalos de despegue de vuelos por ruta (route_no)
utilizando LAG() y OVER()
SELECT fl.flight_id,
       fl.route_no,
       fl.scheduled_departure,
       fl.actual_departure,
       LAG(fl.actual_departure) OVER (
         PARTITION BY fl.route_no
         ORDER BY fl.actual_departure ASC
       ) AS despegue_vuelo_anterior,
       fl.actual_departure - LAG(fl.actual_departure) OVER (
         PARTITION BY fl.route_no
         ORDER BY fl.actual_departure ASC
       ) AS intervalo_con_anterior
FROM bookings.flights fl
WHERE fl.status = 'Arrived'
  AND fl.actual_departure IS NOT NULL
LIMIT 200;
```

A continuación vamos a ver con más detalle las operaciones LAG y LEAD para desplazarnos por los diferentes elementos de la ventana.

Navegación entre Registros: LAG() y LEAD()

PostgreSQL permite realizar recorridos hacia adelante y atrás de manera ordenada dentro de la ventana mediante las funciones de navegación LAG() y LEAD(). Esto evita los costosos y lentos autoparches o JOINs lógicos complejos.



La Fila Actual (Current Row)

- **Representación:** es el bloque central y destacado en azul oscuro la imagen.
- **Concepto:** representa el registro físico de la tabla virtual que PostgreSQL está evaluando en el instante de tiempo de la transacción. Todos los desplazamientos analíticos toman esta fila como el punto de origen (Offset 0).

El Desplazamiento hacia Atrás con LAG()

- **Representación:** en la imagen, se muestra un vector dinámico con dirección ascendente hacia la Fila Anterior (Fila−1) etiquetado con la instrucción LAG().
- **Concepto:** esta función permite al motor de base de datos "mirar hacia atrás" en el tiempo o en el orden establecido. Es sumamente útil para:
  - Calcular diferencias de tiempo entre vuelos consecutivos de una misma aeronave.
  - Analizar variaciones de precios entre el billete actual y el emitido inmediatamente antes en la misma reserva.

El Desplazamiento hacia Adelante con LEAD()

- **Representación:** en la imagen, lo vemos como un vector descendente dirigido hacia la Fila Siguiente (Fila+1) etiquetado con la instrucción LEAD().
- **Concepto:** permite al motor "anticipar" o "mirar hacia adelante" para recuperar un valor de un registro que cronológicamente se procesa después dentro de la ventana lógica. Se emplea habitualmente para:
  - Proyectar la hora del próximo despegue para planificar trabajos de pista en el aeropuerto.
  - Determinar la duración del intervalo hasta el siguiente pedido de un cliente.

Ejemplos prácticos sobre demodb

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

Cálculo del intervalo de tiempo entre vuelos consecutivos de una misma ruta (LAG)

El departamento de operaciones aeroportuarias desea analizar la frecuencia de salida de los vuelos. Necesitamos mostrar, para cada vuelo individual, la fecha del vuelo programado inmediatamente anterior en su misma ruta (route_no) y calcular el intervalo de tiempo exacto que transcurrió entre ambos despegues.

```
-- 1. Vuelos programados y su diferencia con el vuelo anterior de la misma ruta
utilizando LAG() y OVER()
SELECT fl.flight_id,
       fl.route_no,
       fl.scheduled_departure,
       -- Obtener la salida programada del vuelo anterior de la misma ruta
       LAG(fl.scheduled_departure) OVER (
         PARTITION BY fl.route_no
         ORDER BY fl.scheduled_departure ASC
       ) AS salida_vuelo_anterior,
       -- Calcular la diferencia temporal exacta
       fl.scheduled_departure - LAG(fl.scheduled_departure) OVER (
         PARTITION BY fl.route_no
         ORDER BY fl.scheduled_departure ASC
       ) AS intervalo_con_anterior
FROM bookings.flights fl
ORDER BY fl.route_no,
         fl.scheduled_departure;
```

Explicación técnica de la consulta

PARTITION BY fl.route_no: segmenta o agrupa el universo de vuelos en "ventanas" lógicas independientes basadas en cada número de ruta. De este modo, la comparación sólo ocurre entre vuelos de la misma ruta (por ejemplo, nunca se comparará un vuelo Madrid-Barcelona con uno de Barcelona-Londres).

ORDER BY fl.scheduled_departure ASC: ordena cronológicamente los vuelos de cada partición de la ruta, desde el más antiguo hasta el más reciente. Esto es indispensable para que el concepto de "anterior" tenga un sentido cronológico correcto.

LAG(fl.scheduled_departure): accede a la columna scheduled_departure del registro que se encuentra exactamente una posición por detrás (el valor por defecto del desfase u offset es 1) del registro actual en la ventana ordenada. Para el primer vuelo de cada ruta, esta función devolverá un valor nulo (NULL), ya que no hay un predecesor.

fl.scheduled_departure - LAG(...): en PostgreSQL, restar dos marcas de tiempo de tipo timestamp devuelve de manera directa un intervalo (interval), exponiendo de manera legible los días, horas, minutos y segundos de diferencia entre operaciones de aeronaves.

Proyección de salidas y planificación de pista (LEAD)

El equipo de control de tráfico aéreo en pista requiere anticipar la salida del siguiente avión asignado a una ruta para optimizar la asignación de personal de embarque. Se solicita listar los vuelos ordenados cronológicamente mostrando cuándo despegará el siguiente vuelo de esa ruta.

```sql
-- 2. Análisis de vuelos programados y su diferencia con el vuelo siguiente de
la misma ruta utilizando LEAD() y OVER()
SELECT fl.flight_id,
       fl.route_no,
       fl.scheduled_departure,
       fl.status,
       -- Obtener la salida programada del siguiente vuelo de la ruta
       LEAD(fl.scheduled_departure) OVER (
           PARTITION BY fl.route_no
           ORDER BY fl.scheduled_departure ASC
       ) AS salida_siguiente_vuelo,
       -- Calcular cuánto tiempo de margen hay hasta la siguiente salida
       LEAD(fl.scheduled_departure) OVER (
           PARTITION BY fl.route_no
           ORDER BY fl.scheduled_departure ASC
       ) - fl.scheduled_departure AS margen_tiempo_siguiente
FROM   bookings.flights fl
WHERE   fl.status <> 'Cancelled' -- Excluir vuelos cancelados que no operarán
ORDER BY fl.route_no,
         fl.scheduled_departure;
```

Análisis de la evolución de tarifas de billetes por tramo (LAG avanzado)

El departamento de estrategia de precios, quiere auditar la fluctuación de los precios de los billetes vendidos. Para un mismo vuelo (flight_id) y una misma clase de servicio (fare_conditions), se requiere ordenar los tramos de viaje vendidos por su precio de forma ascendente para comparar cada tarifa con la tarifa inmediatamente anterior e identificar la diferencia monetaria exacta.

```
-- 3.Comparación de incremento precios de billetes por clase (fare_conditions)
utilizando LAG() y OVER()
-- Creamos una tabla temporal con precios aleatorios para simular incrementos
de tarifas
-- DROP TABLE IF EXISTS bookings.segments_price_random;
CREATE TABLE bookings.segments_price_random AS
SELECT sg2.flight_id,
        sg2.ticket_no,
        sg2.fare_conditions,
        sg2.price+floor(random() * (100 - 1 + 1) + 1)::int AS price
FROM    bookings.segments sg2
WHERE   sg2.flight_id BETWEEN 2000 AND 5000
  AND   MOD(sg2.flight_id, 2) = 0;
-- Comprobamos la tabla creada
SELECT * FROM bookings.segments_price_random;
-- Ejecutamos la consulta para comparar los precios de billetes por clase y
calcular el incremento con respecto al precio anterior
SELECT sg.flight_id,
        sg.ticket_no,
        sg.fare_conditions,
        sg.price,
        -- Obtener el precio inmediatamente inferior para la misma clase y
vuelo
        LAG(sg.price) OVER (
            PARTITION BY sg.flight_id, sg.fare_conditions
            ORDER BY sg.price ASC
        ) AS precio_anterior,
        -- Calcular la diferencia del incremento de tarifa
        sg.price - LAG(sg.price) OVER (
            PARTITION BY sg.flight_id, sg.fare_conditions
            ORDER BY sg.price ASC
        ) AS incremento_precio
FROM bookings.segments_price_random sg
WHERE sg.fare_conditions IN ('Economy', 'Confort')
  --AND sg.flight_id/2 = 0
ORDER BY sg.flight_id,
          sg.fare_conditions,
          sg.price;
```

pontIA

Optimizando mediante la cláusula WINDOW

Si se van a utilizar múltiples funciones de ventana en una misma consulta analítica compleja y estas comparten criterios de segmentación y ordenación idénticos (como ocurre en los ejemplos anteriores), se puede simplificar la sintaxis utilizando la cláusula WINDOW al final de la consulta para evitar redundancias y optimizar el rendimiento del planificador de PostgreSQL.

Esto le permite a PostgreSQL ordenar los datos una única vez en memoria (work_mem) y reutilizar de manera eficiente ese estado físico para calcular tanto el LAG como el LEAD simultáneamente.

```sql
-- 1. Consulta original: vuelos programados y su diferencia con el vuelo
anterior de la misma ruta utilizando LAG() y OVER()
SELECT fl.flight_id,
       fl.route_no,
       fl.scheduled_departure,
       -- Obtener la salida programada del vuelo anterior de la misma ruta
       LAG(fl.scheduled_departure) OVER (
         PARTITION BY fl.route_no
         ORDER BY fl.scheduled_departure ASC
       ) AS salida_vuelo_anterior,
       -- Calcular la diferencia temporal exacta
       fl.scheduled_departure - LAG(fl.scheduled_departure) OVER (
         PARTITION BY fl.route_no
         ORDER BY fl.scheduled_departure ASC
       ) AS intervalo_con_anterior
FROM bookings.flights fl
ORDER BY fl.route_no,
         fl.scheduled_departure;

-- 2. Consulta optimizada con ventanas reutilizables utilizando WINDOW
SELECT fl.flight_id,
       fl.route_no,
       fl.scheduled_departure,
       LAG(fl.scheduled_departure) OVER w_ruta AS salida_anterior,
       LEAD(fl.scheduled_departure) OVER w_ruta AS salida_siguiente
FROM bookings.flights fl
-- Declaración de ventana con nombre reutilizable
WINDOW w_ruta AS (PARTITION BY fl.route_no ORDER BY fl.scheduled_departure ASC)
ORDER BY fl.route_no,
         fl.scheduled_departure;
```

Anexos

¿Sobrecargan las Window Functions el motor de la base de datos?

Las funciones de ventana pueden generar una sobrecarga muy significativa de recursos (CPU, memoria y disco) si la consulta o la arquitectura de la base de datos no están optimizadas.

La sobrecarga física del motor se debe principalmente a los siguientes factores:

- **Exigencia de datos preordenados:** para poder calcular métricas particionadas o de navegación, las funciones de ventana siempre requieren que los datos estén previamente ordenados en memoria. El ordenamiento (sorting) es una de las operaciones computacionales más costosas dentro de un motor de bases de datos relacionales.
- **Múltiples pases de ordenamiento:** si una misma consulta utiliza múltiples funciones de ventana con criterios de PARTITION BY o ORDER BY diferentes, el motor de base de datos no puede compartir el estado ordenado de los datos. Por este motivo, el planificador se verá obligado a inyectar pasos de ordenamiento adicionales entre las diferentes fases de evaluación del plan de ejecución.
- **Vuelcos de memoria a disco:** cuando el volumen de datos que debe ser ordenado para calcular la ventana supera la cantidad de memoria física asignada al parámetro work_mem, PostgreSQL no puede completar la ordenación en RAM. En su lu-

gar, el motor tiene que escribir y leer bloques de datos en archivos temporales de disco, lo que degrada críticamente el rendimiento y la latencia de la consulta.

Elementos para evitar la sobrecarga y optimizar las consultas

Para prevenir la degradación del rendimiento al utilizar funciones de ventana, PostgreSQL ofrece varios elementos sintácticos y de configuración que deben coordinarse de forma estricta:

A. Reutilización del orden con particiones idénticas

PostgreSQL garantiza que todas las funciones de ventana que tengan especificaciones idénticas de PARTITION BY y ORDER BY verán los datos en el mismo orden de ejecución. Esto permite al motor realizar un único ordenamiento y compartir el estado entre todas las funciones, eliminando pases de CPU adicionales. Es una gran práctica alinear los criterios de las ventanas siempre que sea posible.

B. La cláusula WINDOW

Para evitar tener que escribir repetidamente cláusulas OVER idénticas en el SELECT, lo cual puede provocar errores de mantenimiento y duplicar análisis, podemos utilizar la cláusula WINDOW. Esta cláusula permite definir el comportamiento y la estructura de la ventana al final de la consulta, asignarle un nombre y luego referenciarla de forma limpia en el SELECT

```
SELECT sum(salary) OVER w, avg(salary) OVER w
FROM empsalary
WINDOW w AS (PARTITION BY depname ORDER BY salary DESC);
```

C. Filtrado temprano con WHERE

Dado que el volumen de datos procesado por la ventana depende directamente del resultado de la tabla virtual previa, aplicar filtros restrictivos en la cláusula WHERE (apoyados por índices adecuados) reduce de forma drástica el número de registros que PostgreSQL tendrá que ordenar.

D. Filtrado posterior mediante Subconsultas (Sub-selects)

Las funciones de ventana están estrictamente prohibidas en las cláusulas WHERE, GROUP BY y HAVING de la misma consulta. Si es necesario filtrar filas sobre un cálculo de una ventana (por ejemplo, si usamos row_number() para paginar y solo necesitamos los primeros 3 registros por grupo), se debe encapsular la consulta que calcula la ventana dentro de una subconsulta y aplicar el filtro en la consulta exterior. Esto detiene la propagación de datos innecesarios de forma eficiente:

```
SELECT depname, empno, salary
FROM (
    SELECT depname, empno, salary,
           row_number() OVER (PARTITION BY depname ORDER BY salary DESC) AS pos
    FROM empsalary
) AS ss
WHERE pos < 3; -- Filtro de ventana óptimo mediante subconsulta
```

E. Optimización de la memoria con work_mem

Para consultas analíticas complejas o reportes masivos que involucren funciones de ventana pesadas, se debe evaluar y ajustar el parámetro work_mem a nivel de transacción o sesión. Asegurar que este límite sea lo suficientemente alto evitará que PostgreSQL vuelque las operaciones de ordenamiento de las ventanas al disco físico, manteniendo la ejecución puramente en memoria RAM de alta velocidad.

Modificación del parámetro de optimización de memoria work_men

El parámetro correcto en PostgreSQL se llama work_mem. Define la cantidad de memoria RAM dedicada a operaciones de ordenamiento (ORDER BY, DISTINCT) y uniones (JOIN) antes de escribir archivos temporales en el disco.

La forma de modificarlo depende del alcance que se desee dar al cambio, ya que no requiere reiniciar el servidor:

1. Solo para la consulta o transacción actual (Recomendado): es la forma más segura. El cambio solo afecta a lo que se ejecute inmediatamente y se destruye al terminar la transacción.

```
BEGIN;
SET LOCAL work_mem = '64MB';
-- Ejecuta tu query pesada aquí
COMMIT;
```

2. Solo para la sesión actual: afecta únicamente a la conexión abierta. Al cerrar la terminal o desconectar la aplicación, vuelve a su valor original.

```
SET work_mem = '32MB';
```

3. Para un usuario (Role) o Base de Datos en específico: cada vez que ese usuario se conecte, o cualquier sesión interactúe con esa base de datos, tendrá ese valor configurado.

```
-- Por usuario
ALTER ROLE mi_usuario SET work_mem = '16MB';

-- Por Base de Datos
ALTER DATABASE mi_base_datos SET work_mem = '16MB';
```

4. De forma global para todo el servidor (Permanente): cambia el valor para todas las conexiones del clúster de forma definitiva. Se puede hacer mediante SQL sin reiniciar la base de datos:

```
-- 1. Aplicar el cambio global
ALTER SYSTEM SET work_mem = '16MB';

-- 2. Recargar la configuración para que tenga efecto (No interrumpe el servicio)
SELECT pg_reload_conf();
```

Nota: también se puede editar directamente el archivo de configuración postgresql.conf, buscar la línea work_mem, cambiar su valor y ejecutar pg_ctl reload desde la consola del sistema operativo.

Advertencia antes de subir el valor

El parámetro work_mem se asigna por operación de consulta, no por usuario conectado. Si una consulta ejecuta tres ordenamientos concurrentes y se ha marcado work_mem = '64MB', esa sola consulta consumirá 192MB. Si en la aplicación hay 100 usuarios simultáneos, se puede agotar rápidamente la memoria RAM del servidor (provocando un error Out of Memory).

Si se ha modificando work_mem porque las consultas van lentas, hay que tratar de detectar si se han producido escrituras en archivos temporales (temp_files). Esto nos podría estar indicando que la memoria asignada a la consulta es insuficiente o que hemos superado el umbral de memoria que el servidor puede asignar a la consulta.